

COP 3515 – Advanced Program Design

Semester Programming Project

Client Change Request (CCR-005)

Project: Student Information Management System (SIMS)

Customer: University Registrar's Office

Priority: High

Assigned To: Student Developer

Sprint: Week 5

Due Date: Beginning of Lesson 12

Business Background

The University Registrar's Office has been extremely satisfied with the progress of the Student Information Management System (SIMS). The application now records student information, stores course grades, determines academic standing, and saves student records to a text file.

However, employees have identified one significant usability issue. Each time they want to perform a different task, they must restart the program. For example, after entering student information they must restart the application to display the student record or save it to a file. This is inconvenient and slows down daily operations.

The Registrar would like the application to remain running until the employee chooses to exit. Employees should be able to perform multiple operations during a single execution of the program through a simple menu-driven interface.

This enhancement should be added to the existing SIMS application developed during **CCR-001 through CCR-004. Do not create a new program.** Continue enhancing your existing codebase.

Business Requirements

The Registrar would like the application to continue performing **all functionality from CCR-001 through CCR-004**.

In addition, the application should display a menu similar to the following:

Student Information Management System

1. Add Student
2. Display Student
3. Enter Grades
4. Save Student Record
5. Exit

Selection:

The application should continue displaying the menu until the employee chooses **Exit**.

The application should allow the employee to perform multiple operations during one execution without restarting the software.

At a minimum, the menu shall support:

1. Add Student
2. Display Student
3. Enter Grades
4. Save Student Record
5. Exit Program

When the user selects **Exit**, the application should terminate normally.

If the user enters an invalid menu option, an understandable error message should be displayed, and the menu should be shown again.

Acceptance Criteria

The project will be considered complete when all of the following requirements have been met.

- ☐ The program compiles without errors or warnings.
 - ☐ All functionality from **CCR-001** continues to work correctly.
 - ☐ All functionality from **CCR-002** continues to work correctly.
 - ☐ All functionality from **CCR-003** continues to work correctly.
 - ☐ All functionality from **CCR-004** continues to work correctly.
 - ☐ The menu is displayed repeatedly.
 - ☐ The program does not terminate until **Exit** is selected.
 - ☐ Each menu option performs the requested task.
 - ☐ Invalid menu selections are handled gracefully.
 - ☐ Program exits normally when requested.
-

Technical Constraints

For this week's assignment, you may **only** use C language features covered through **Lesson 12**.

Permitted concepts include:

- Variables
- Constants
- Arrays
- Enumerations
- User input
- Formatted output
- File processing
- Character processing
- while loops

- for loops
- break
- continue

Do **not** use language features that have not yet been introduced in class.

Out of Scope

The Registrar is **not** requesting any of the following features at this time.

- Multiple students
- Searching student records
- Editing existing student information
- Deleting student records
- Dynamic memory allocation
- Databases
- Graphical user interfaces (GUI)
- Password protection

These features will be requested in future Client Change Requests.

Questions for the Customer

Before writing any code, identify questions you would ask the customer to clarify the requirements.

Examples include:

1. Which menu option should appear when the program first starts?
2. Should the menu be redisplayed after every operation?
3. What should happen if the employee selects **Display Student** before adding a student?
4. What should happen if **Save Student Record** is selected before student information has been entered?

5. Should the program ask the employee to confirm before exiting?
 6. What should happen if the employee enters a letter instead of a number?
 7. Should invalid menu selections count as an error or simply redisplay the menu?
 8. Should the program automatically save information before exiting?
 9. Should menu numbering remain fixed in future versions?
 10. Will future versions include additional menu options?
 11. Should menu selections be case-sensitive if letters are ever introduced?
 12. Should the screen be cleared before redisplaying the menu?
-

Example Program Execution

Program Startup

Student Information Management System

Version 5.0

1. Add Student
2. Display Student
3. Enter Grades
4. Save Student Record
5. Exit

Selection:

Example 1 – Add Student

User Input

Selection: 1

Student ID: 1234567

Student Name: Alice Johnson

Current GPA: 3.84

Expected Output

Student successfully added.

Example 2 – Enter Grades

User Input

Selection: 3

95

88

91

84

100

Expected Output

Grades successfully recorded.

Example 3 – Display Student

User Input

Selection: 2

Expected Output

Student Summary

Student ID : 1234567

Student Name : Alice Johnson

Current GPA : 3.84

Academic Standing : Honors

Course Grades

95

88

91

84

100

Average Grade : 91.60

Highest Grade : 100

Lowest Grade : 84

Example 4 – Save Student Record

User Input

Selection: 4

Expected Output

Student record successfully saved.

student_records.txt created.

Example 5 – Invalid Menu Selection

User Input

Selection: 9

Expected Output

ERROR

Invalid menu selection.

Please choose an option between 1 and 5.

Example 6 – Exit

User Input

Selection: 5

Expected Output

Thank you for using the

Student Information Management System.

Program terminated successfully.

Regression Testing Requirements

Since the Student Information Management System is developed incrementally, **every previously implemented feature must continue to operate correctly.**

Your submission must demonstrate that:

- CCR-001 functionality still works.
- CCR-002 functionality still works.
- CCR-003 functionality still works.
- CCR-004 functionality still works.
- CCR-005 functionality works correctly.

At a minimum, include the following regression tests.

Test Description	Expected Result
1 Add a student	Student information stored successfully
2 Enter course grades	Grades recorded correctly
3 Display student information	Student report displayed correctly
4 Save student record	File created successfully
5 Enter an invalid menu option	Error message displayed and menu redisplayed
6 Perform several menu operations without restarting the program	All operations complete successfully

Test Description

Expected Result

7 Exit the program

Program terminates normally

For **each** regression test include:

- Test description
- Input
- Expected output
- Actual output
- Pass/Fail result

If a defect is discovered:

1. Document the defect.
 2. Correct the defect.
 3. Repeat the affected test.
 4. Verify that all previously implemented functionality continues to work correctly.
-

AI Usage Policy

Artificial Intelligence tools (such as ChatGPT, GitHub Copilot, Claude, or Gemini) may be used during this assignment.

However, you remain responsible for:

- Understanding every line of submitted code.
- Testing all functionality.
- Correcting any defects.
- Ensuring that new functionality does not break existing features.

Software engineering is not simply writing code—it is delivering **working software**.

Submission Requirements

Submit a single ZIP file containing:

- main.c
- Any additional source files (if applicable)
- student_records.txt (generated by your program)

Also include a PDF containing:

- Screenshots demonstrating each menu option
 - Screenshots showing successful regression testing
 - A brief description of any bugs encountered and how they were corrected
 - Answers to the **Questions for the Customer**
 - Any assumptions made because the customer requirements were ambiguous
-

Looking Ahead

This is the **fifth** release of the Student Information Management System (SIMS).

Future Client Change Requests will continue to expand the application by introducing **functions, modular program design, pointers, dynamic memory allocation**, and additional software engineering concepts. As the application grows, the menu system developed in this assignment will serve as the primary interface through which users access new functionality.

Remember that **professional software is built incrementally**. Each Client Change Request adds new business value while preserving existing capabilities. A successful solution demonstrates not only that the new feature works, but also that every feature delivered in previous releases continues to operate correctly through comprehensive regression testing.